

Launch one EC2 using Amazon Linux 2 image and add a script in user data to install Apache.

Objective

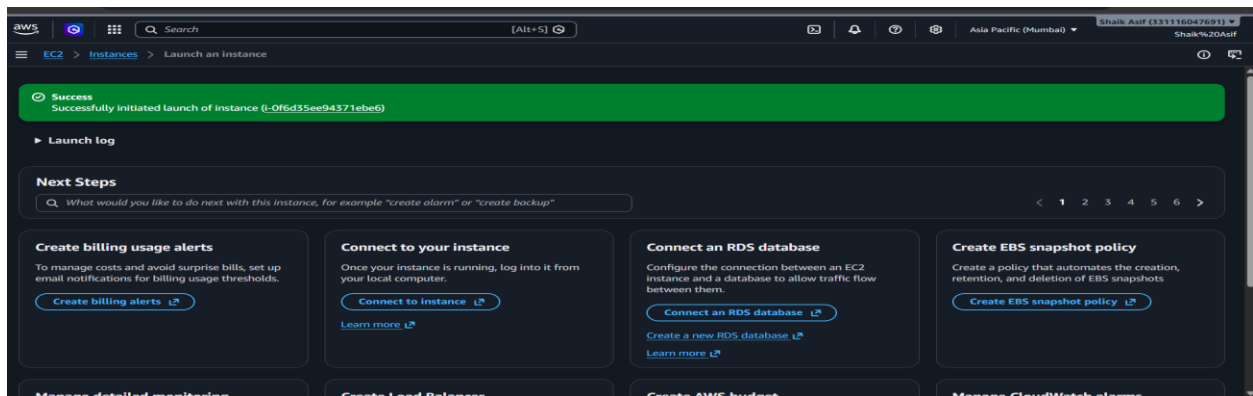
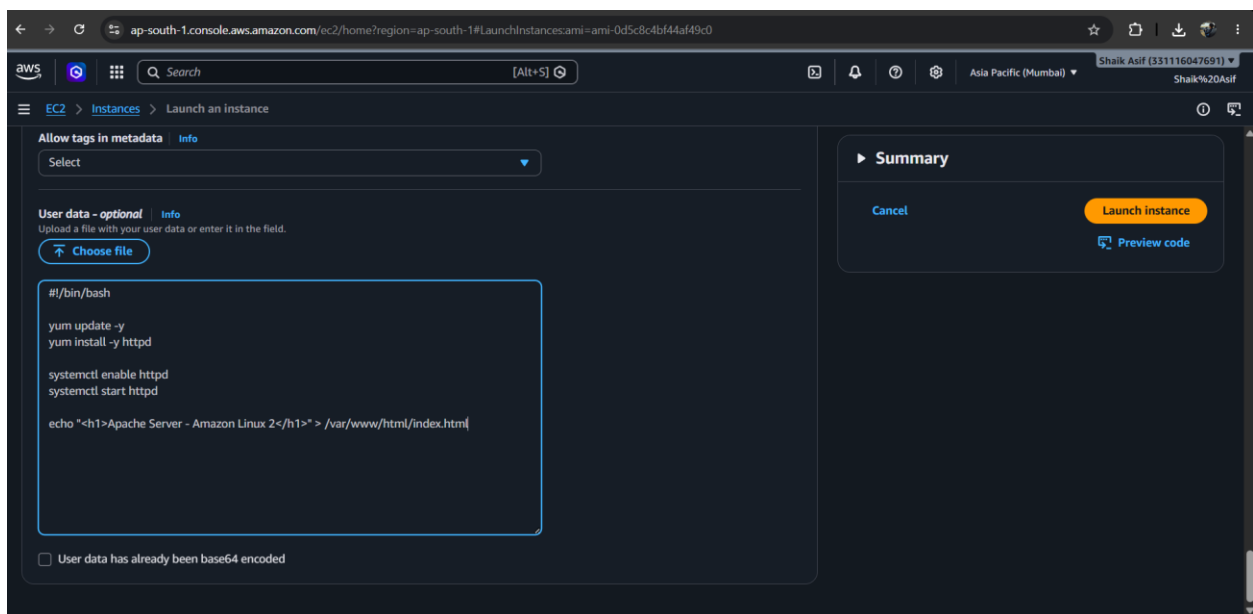
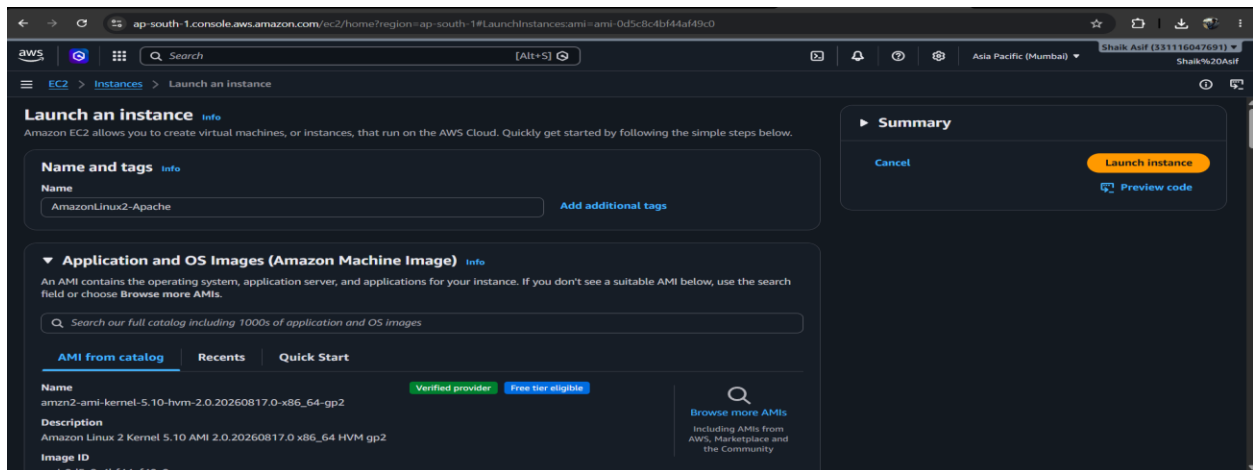
To launch an Amazon Linux 2 EC2 instance and install Apache using User Data.

Steps

1. Open **AWS Console** → **EC2** → **Launch Instance**.
2. Select the **Amazon Linux 2 AMI**.
3. Select the required instance type.
4. Select/create a **Key Pair**.
5. Configure the Security Group:
 - a. Allow **SSH – Port 22**
 - b. Allow **HTTP – Port 80**
6. Go to **Advanced Details** → **User Data**.
7. Enter the following script:

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl enable httpd
systemctl start httpd
echo "<h1>Apache Server - Amazon Linux 2</h1>" > /var/www/html/index.html
```

8. Click **Launch Instance**.
9. Wait until the instance status shows **Running**.
10. Copy the **Public IPv4 address**.





```
asifs@Asif-Shaik MINGW64 ~/downloads (master)
$ ssh -i amazon_task.pem ec2-user@13.203.222.20
The authenticity of host '13.203.222.20 (13.203.222.20)' can't be established.
ED25519 key fingerprint is SHA256:epUY70AZroRQU7H/m5QMtb3CtZvylemITZQ2t9LUAUU.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.203.222.20' (ED25519) to the list of known hosts.

#####
#
# Amazon Linux 2
#
#####
AL2 End of Life is 2026-06-30.

A newer version of Amazon Linux is available!

Amazon Linux 2023, GA and supported until 2029-06-30.
https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-172-31-0-181 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; vendor preset: disabled)
   Active: active (running) since Tue 2026-08-25 10:33:37 UTC; 5min ago
     Docs: man:httpd.service(8)
  Main PID: 2564 (httpd)
    Status: "Total requests: 2; Idle/Busy workers 100/0;Requests/sec: 0.00627; Bytes served/sec: 5 B/sec"
    CGroup: /system.slice/httpd.service
            └─2564 /usr/sbin/httpd -DFOREGROUND
              └─2565 /usr/sbin/httpd -DFOREGROUND
                └─2566 /usr/sbin/httpd -DFOREGROUND
                  └─2567 /usr/sbin/httpd -DFOREGROUND
                    └─2568 /usr/sbin/httpd -DFOREGROUND
                      └─2569 /usr/sbin/httpd -DFOREGROUND
                        └─2625 /usr/sbin/httpd -DFOREGROUND
                          └─2628 /usr/sbin/httpd -DFOREGROUND
                            └─2631 /usr/sbin/httpd -DFOREGROUND

Aug 25 10:33:37 ip-172-31-0-181.ap-south-1.compute.internal systemd[1]: Starting The Apache HTTP Server...
Aug 25 10:33:37 ip-172-31-0-181.ap-south-1.compute.internal systemd[1]: Started The Apache HTTP Server.
[ec2-user@ip-172-31-0-181 ~]$ curl http://localhost
sh1>Apache Server - Amazon Linux 2</h1>
```

Result

Task 1 completed successfully. Apache was installed automatically using EC2 User Data and the web page was accessed through the EC2 public IP.

Launch one EC2 using Ubuntu image and add a script in user data to install Nginx.

Objective

To launch an Ubuntu EC2 instance and install Nginx automatically using User Data.

Steps

1. Open **AWS Console** → **EC2** → **Launch Instance**.
2. Enter the instance name as **Ubuntu-Nginx**.
3. Select **Ubuntu Server 24.04 LTS** as the AMI.
4. Select the required instance type.
5. Select the **Key Pair**.
6. Configure the Security Group:
 - a. Allow **SSH – Port 22 – My IP**
 - b. Allow **HTTP – Port 80 – Anywhere**
7. Enable **Auto-assign Public IP**.
8. Go to **Advanced Details** → **User Data**.
9. Enter the following script:

```
#!/bin/bash
```

```
apt-get update -y  
apt-get install -y nginx
```

```
systemctl enable nginx  
systemctl start nginx
```

```
echo "<h1>Nginx Server - Ubuntu</h1>" > /var/www/html/index.html
```

10. Click **Launch Instance**.
11. Wait until the instance status shows **Running**.
12. Copy the **Public IPv4 address**.

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Shaik Asif (331116047691)

Shaik%20Asif

EC2

Instances

Launch an instance

Launch an instance

Info

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags

Info

Name

Ubuntu-Nginx

Add additional tags

▼ Application and OS Images (Amazon Machine Image)

Info

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Q Search our full catalog including 1000s of application and OS images

Recents

Quick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

Ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Linux

SUSE Linux

Debian

Debian

Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

▼ Summary

Number of instances

Info

1

Software image (AMI)

Canonical, Ubuntu, 24.04, amd64...[read more](#)

ami-006f82a1d5a27da54

Virtual server type (instance type)

t3.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Cancel

Launch instance

Preview code

User data - optional

Info

Upload a file with your user data or enter it in the field.

Choose file

```
#!/bin/bash

apt-get update -y
apt-get install -y nginx

systemctl enable nginx
systemctl start nginx

echo "<h1>Nginx Server - Ubuntu</h1>" > /var/www/html/index.html
```



Result

Task 2 completed successfully. Nginx was installed automatically on the Ubuntu EC2 instance using User Data.

Launch one Windows server and install Tomcat on Windows.

. Objective

To launch a Windows Server EC2 instance in AWS, connect to it using RDP, install Java JDK 17, install Apache Tomcat 10.1.59, configure the required environment variables, and verify that Tomcat is running successfully.

2. Steps / Implementation

Step 1: Launch Windows EC2 Instance

1. Log in to the **AWS Management Console**.
2. Open **EC2**.
3. Click **Launch Instance**.
4. Enter the instance name:

Windows-Tomcat

5. Select a **Windows Server AMI**.
6. Select the required instance type.
7. Select the key pair.
8. Configure the Security Group.
9. Allow **RDP – TCP 3389** from your IP address.
10. Launch the instance.
11. Wait until the instance status shows **Running**.

Step 2: Connect to Windows EC2 Using RDP

1. Select the Windows EC2 instance.
2. Click **Connect**.
3. Select **RDP client**.
4. Click **Get password**.
5. Upload the .pem key file.
6. Decrypt and copy the Administrator password.
7. Download the Remote Desktop file.
8. Open the .rdp file.
9. Log in using:

Username: Administrator

10. Enter the decrypted password.

The Windows Server desktop was opened successfully.

Step 3: Install Java JDK 17

Java was installed on the Windows EC2 server because Tomcat requires Java.

After installation, open **Command Prompt** and run:

```
java -version
```

Expected result:

```
java version "17..."
```

Verify the Java location:

```
where java
```

The JDK 17 installation was available under:

```
C:\Program Files\Java\jdk-17
```

Step 4: Set JAVA_HOME

Set the Java environment variable:

```
JAVA_HOME=C:\Program Files\Java\jdk-17
```

Verify it using:

```
echo %JAVA_HOME%
```

Expected result:

```
C:\Program Files\Java\jdk-17
```

Step 5: Download and Extract Tomcat

1. Download **Apache Tomcat 10.1.59** for Windows.
2. Extract the Tomcat ZIP file.
3. Create/use the following directory:

```
C:\Tomcat
```

4. Place the Tomcat files inside the Tomcat directory.

The Tomcat installation path used was:

C:\Tomcat\apache-tomcat-10.1.59

The important Tomcat directories were:

C:\Tomcat\apache-tomcat-10.1.59\bin

C:\Tomcat\apache-tomcat-10.1.59\conf

C:\Tomcat\apache-tomcat-10.1.59\logs

C:\Tomcat\apache-tomcat-10.1.59\webapps

Step 6: Configure Tomcat Environment Variables

The Tomcat environment variables were configured as follows:

CATALINA_BASE=C:\Tomcat\apache-tomcat-10.1.59
CATALINA_HOME=C:\Tomcat\apache-tomcat-10.1.59
JRE_HOME=C:\Program Files\Java\jdk-17

The Tomcat temporary directory was:

C:\Tomcat\apache-tomcat-10.1.59\temp

Step 7: Open Tomcat bin Directory

Open Command Prompt and run:

```
cd C:\Tomcat\apache-tomcat-10.1.59\bin
```

Check the files:

```
dir
```

The startup.bat file should be available in the bin directory.

Step 8: Start Tomcat

Run:

```
startup.bat
```

Tomcat starts successfully.

Alternatively:

```
catalina.bat run
```

can be used to run Tomcat in the current Command Prompt window.

Step 9: Verify Tomcat

Open a browser on the Windows server and enter:

<http://localhost:8080>

The **Apache Tomcat welcome page** should appear.

To check whether port 8080 is listening, run:

```
netstat -ano | findstr :8080
```

A listening entry for port 8080 confirms that Tomcat is running.

Step 10: Verify Tomcat from EC2 Public IP

If port 8080 is allowed in the EC2 Security Group, Tomcat can also be accessed from the local computer using:

<http://<EC2-Public-IP>:8080>

3. Result

Successfully launched a Windows Server EC2 instance and connected to it using RDP. Java JDK 17 was installed and configured. Apache Tomcat 10.1.59 was extracted under C:\Tomcat\apache-tomcat-10.1.59, and the required CATALINA_HOME, CATALINA_BASE, and JRE_HOME environment variables were configured. Tomcat was started successfully using startup.bat and verified on port 8080 through the Apache Tomcat welcome page.

4. Evidence / Screenshots

Add these screenshots to the task completion document:

- 1. Windows EC2 instance – Running**
- 2. RDP connection to Windows Server**
- 3. Java JDK 17 version**

```
java -version
```

- 4. JAVA_HOME verification**

```
echo %JAVA_HOME%
```

5. Tomcat folder

C:\Tomcat\apache-tomcat-10.1.59

6. Tomcat environment variables

- a. CATALINA_HOME
- b. CATALINA_BASE
- c. JRE_HOME

7. Tomcat bin folder showing startup.bat

8. Tomcat startup

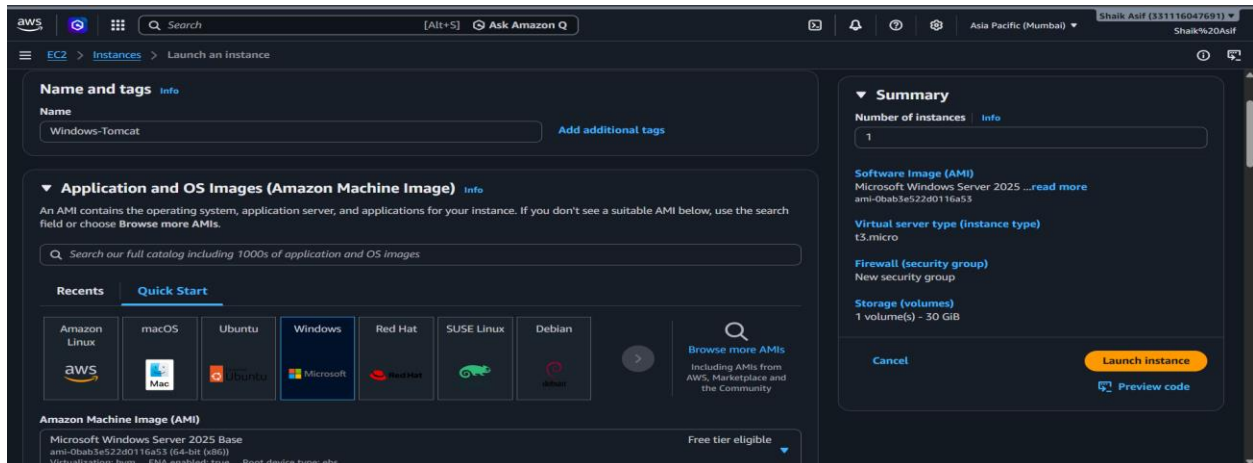
startup.bat

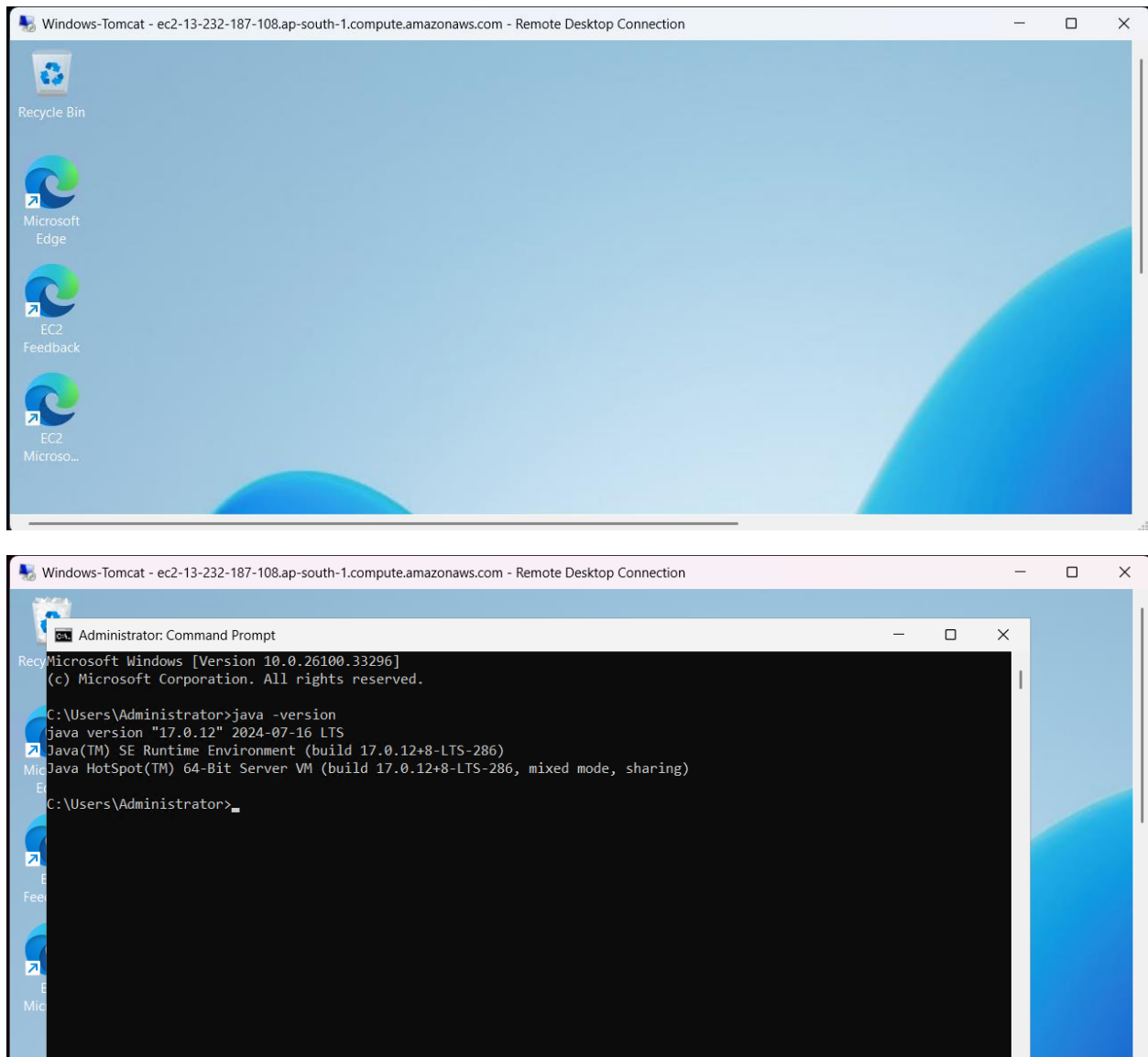
9. Port 8080 verification

```
netstat -ano | findstr :8080
```

10. Apache Tomcat welcome page

http://localhost:8080





Conclusion

The Windows EC2 server was successfully launched and configured with **JDK 17** and **Apache Tomcat 10.1.59**. Tomcat was successfully started and verified through **port 8080**, completing the Windows server and Tomcat installation task.

Take a snapshot of the instance created in Task 1.

Steps

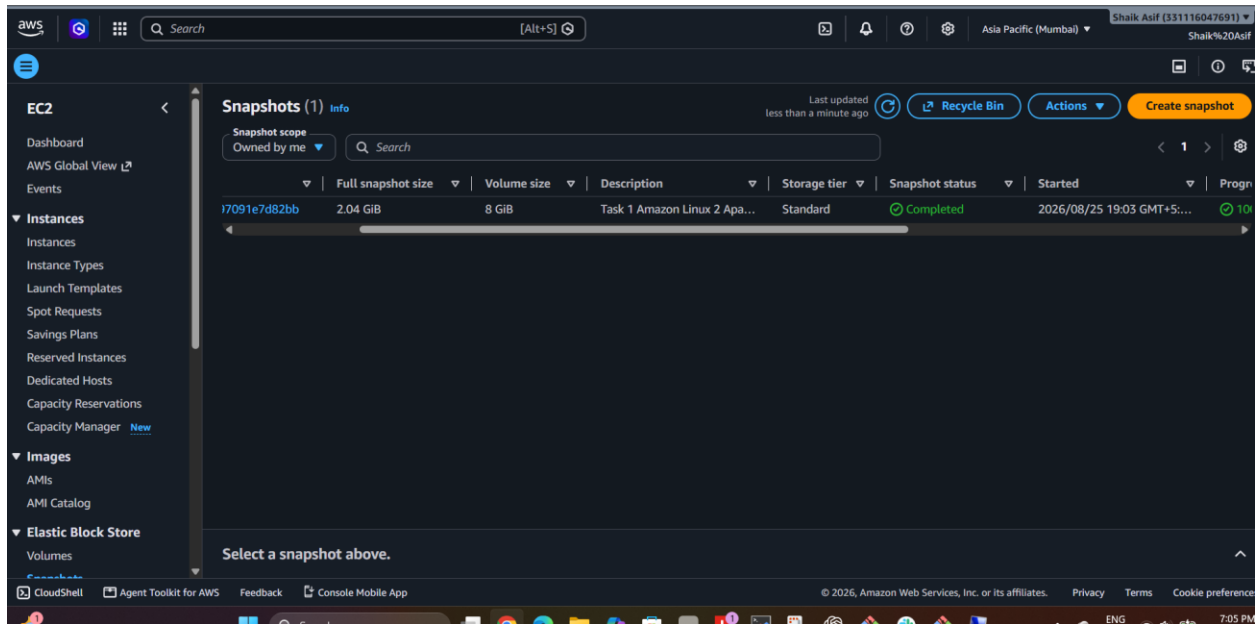
1. Open **AWS Management Console** → **EC2**.
2. Go to **Instances**.
3. Select the **Amazon Linux 2 EC2 instance** created in Task 1.
4. Open the **Storage** tab.
5. Under **Block devices**, click the attached **Volume ID**.
6. In the **Volumes** page, select the volume.
7. Click **Actions** → **Create snapshot**.
8. Enter the description: Task 1 Amazon Linux 2 Apache Snapshot
9. Click **Create snapshot**.
10. Go to **Elastic Block Store** → **Snapshots**.
11. Select the newly created snapshot.
12. Wait until the **Status** becomes **Completed**.

The screenshot shows the AWS Management Console interface for creating a snapshot. The breadcrumb navigation at the top reads: EC2 > Volumes > vol-07d0e91d11a2dceb4 > Create snapshot. The main heading is 'Create snapshot' with an 'Info' icon. Below the heading is a sub-header: 'Create a point-in-time snapshot to back up the data on an Amazon EBS volume to Amazon S3.'

The form is divided into three sections:

- Source volume:** Contains two fields: 'Volume ID' with the value 'vol-07d0e91d11a2dceb4' and 'Availability Zone' with the value 'aps1-az3 (ap-south-1b)'.
- Snapshot details:** Contains a 'Description' field with the text 'Task 1 Amazon Linux 2 Apache Snapshot' and a note '255 characters maximum.' Below it, the 'Encryption' status is 'Not encrypted'.
- Tags:** A section with a note: 'No tags associated with the resource.'

The top of the console shows the AWS logo, a search bar, and the user's profile 'Shaikh Asif (331116047691)' in the 'Asia Pacific (Mumbai)' region.



Result

Successfully created an EBS snapshot of the Amazon Linux 2 EC2 instance created in Task 1. The snapshot status was verified as Completed.

Assign password less authentication for the EC2 created in Task 2.

Steps

1. Open **Git Bash** on your local Windows system.
2. Connect to the **Ubuntu EC2 instance from Task 2** using the .pem key:ssh -i "your-key.pem" ubuntu@YOUR_PUBLIC_IP
3. Create the SSH directory:mkdir -p ~/.ssh
4. Set the directory permission:chmod 700 ~/.ssh
5. On your local Git Bash, display your public SSH key:cat ~/.ssh/id_rsa.pub
6. Copy the complete public key.
7. Back inside the Ubuntu EC2, open:nano ~/.ssh/authorized_keys
8. Paste the public key into the file.
9. Save the file:
Ctrl + O → Enter → Ctrl + X
10. Set the required permission:chmod 600 ~/.ssh/authorized_keys
11. Exit the EC2:exit
12. From your local Git Bash, connect again **without the .pem file**:ssh ubuntu@YOUR_PUBLIC_IP
13. Verify the login:whoamiExpected output:ubuntu

```
$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/c:/Users/asifs/.ssh/id_rsa):
/c:/Users/asifs/.ssh/id_rsa already exists.
Overwrite (y/n)?

asifs@Asif-Shaik MINGW64 ~
$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGD1jK3oqTfj7foTy9j+zC1SSxkgLxFl0IJ1EnzsKYP
CSv69g+YDV4wD1Vc1EBs8hMPjhl7xcLGzP8Xt11ZrxEL7f2eGTYwXJA2KunLMQamKVH6YfHLSVCixGv
OYmjTTC2P7StilfwWJ/11htBzRuNRRG626zMJ9Scdf9qFAmpUaCSs8jTDuzd7+CmnzihgyxDQ3C02FR
FPpbotySgRwIMTBKLGdMBUPVSGZAQdx96agnk4HUzmQdhr/5jDyeq0dQEyebVcs+0J8wrX3vYHuTVW
OwUNFtrZ2Ro17GFlgsFwG13PpqiWedy+u+2T8d51vR/YbreJNcgXmf8IQcy9xw5TB0mh4XZN+cg4
P020bghDpPKPbjhl5/uumsiTuGwYgTd5pPYaysQSBYy+4Gh8qaybMwoaw0n5WIqJR1jzFN6CKWDSbSYM
KpUCJIIjvb/EkZjwlgrQdUPx7yN/qjvH8xdn3tct1RGQ8IbTLY5Mo1DYDTgLo8+4qCINqKM= asifs@A
sif-Shaik

asifs@Asif-Shaik MINGW64 ~
$ cd ~/downloads

asifs@Asif-Shaik MINGW64 ~/downloads (master)
$ ssh -i ubuntu_task.pem ubuntu@13.201.191.43
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1017-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Tue Aug 25 14:30:16 UTC 2026

System load: 0.0          Temperature: ~273.1 C
Usage of /: 29.9% of 6.71GB Processes: 114
Memory usage: 26%        Users logged in: 0
Swap usage: 0%           IPv4 address for ens5: 172.31.3.153

Expanded Security Maintenance for Applications is not enabled.

107 updates can be applied immediately.
91 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-3-153:~$ mkdir -p ~/.ssh
chmod 700 ~/.ssh
```

```
asifs@Asif_Shaik MINGW64 ~/downloads (master)
$ ssh ubuntu@13.201.191.43
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1017-aws x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Tue Aug 25 14:34:27 UTC 2026

System load:  0.32           Temperature:   -273.1 C
Usage of /:   29.9% of 6.71GB Processes:      116
Memory usage: 27%           Users logged in: 0
Swap usage:   0%            IPv4 address for ens5: 172.31.3.153

Expanded Security Maintenance for Applications is not enabled.

107 updates can be applied immediately.
91 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Tue Aug 25 14:30:16 2026 from 115.98.47.116
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-3-153:~$ whoami
ubuntu
```

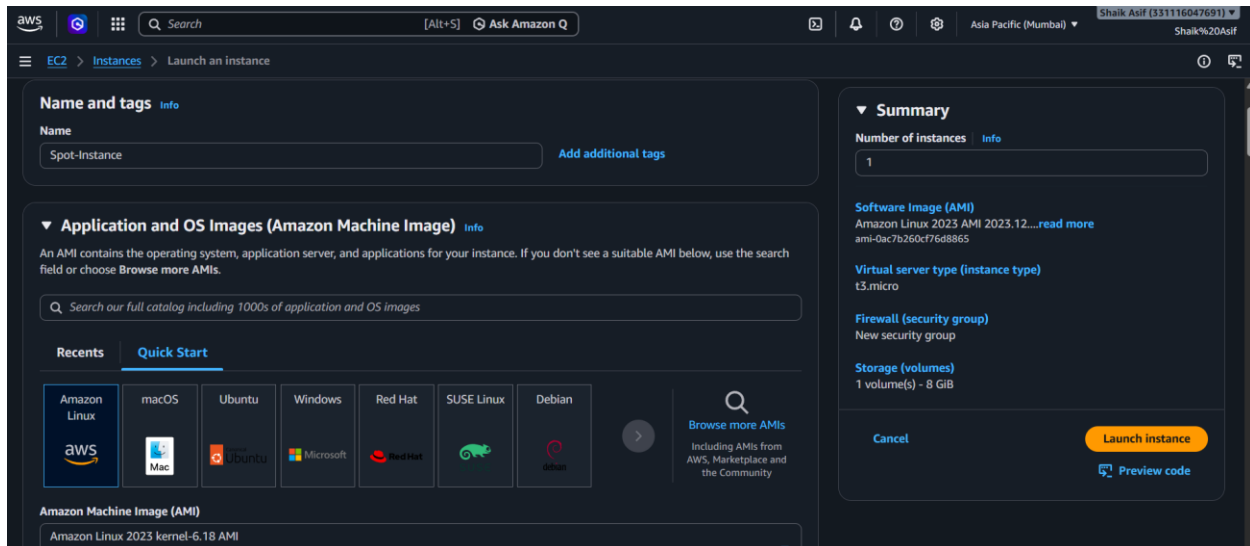
Result

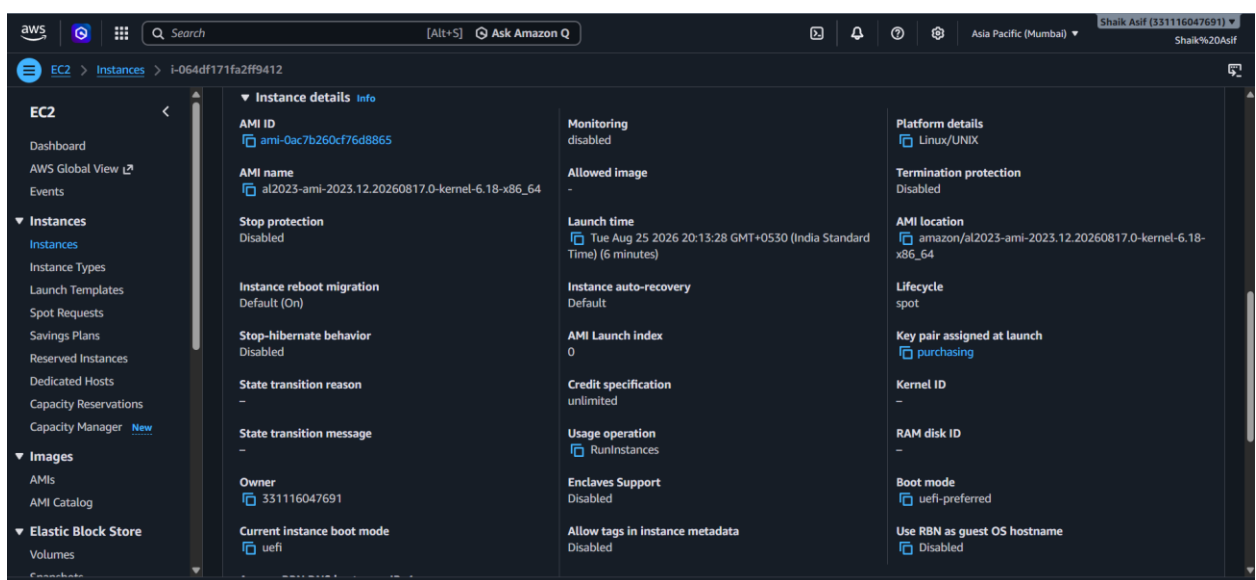
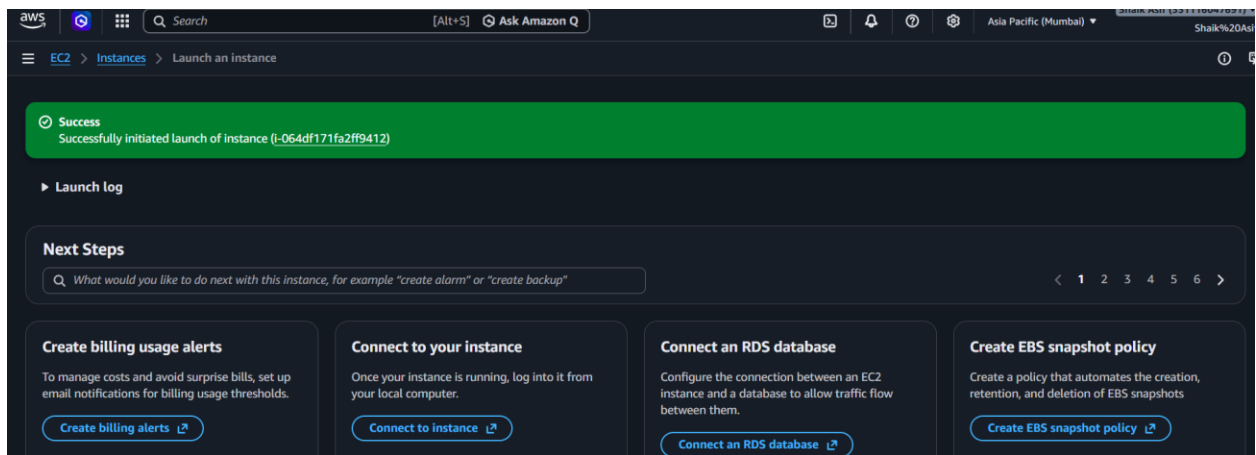
Successfully configured passwordless SSH authentication for the Ubuntu EC2 instance created in Task 2. The connection was verified without specifying the .pem key file

Launch any EC2 using the spot purchasing option.

Steps

1. Open **AWS Management Console** → **EC2**.
2. Click **Launch instance**.
3. Enter an instance name, for example: **Spot-Instance**
4. Select an available **Amazon Linux** AMI.
5. Select an instance type, for example: **t3.micro**
6. Select an existing **Key pair**.
7. Under **Advanced details**, find **Purchasing option**.
8. Select:
Request Spot Instances.
9. Configure the required network and security group settings.
10. Click **Launch instance**.
11. Go to **EC2** → **Instances**.
12. Select the newly launched instance.
13. Verify that the **Lifecycle** is shown as:
spot





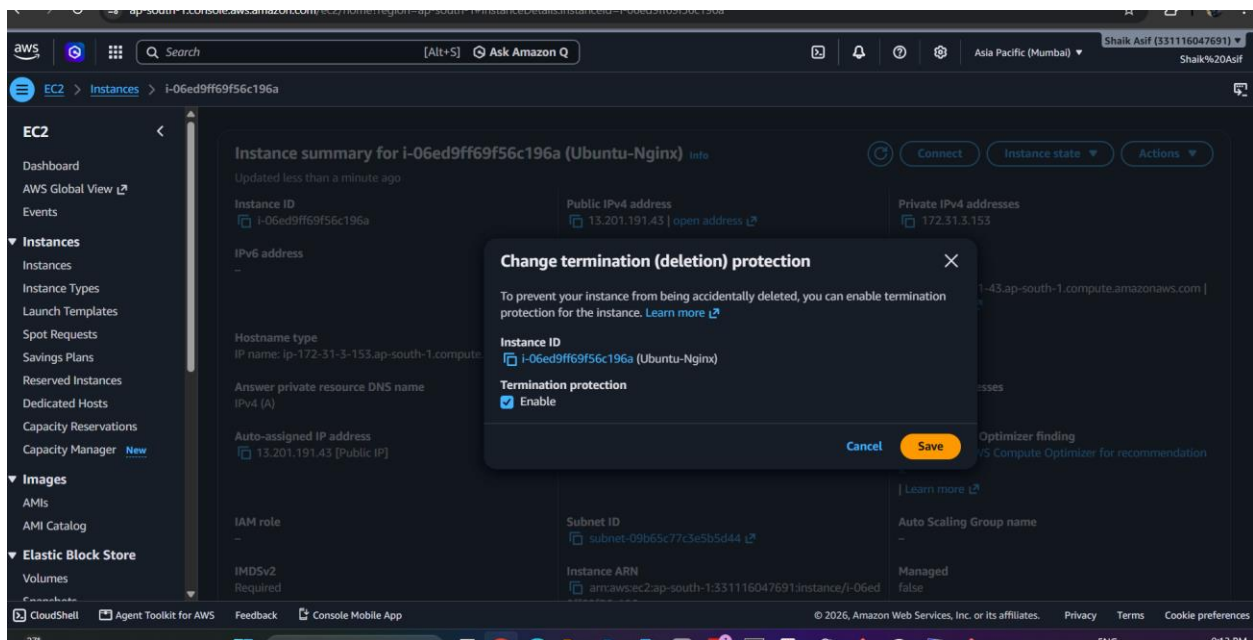
Result

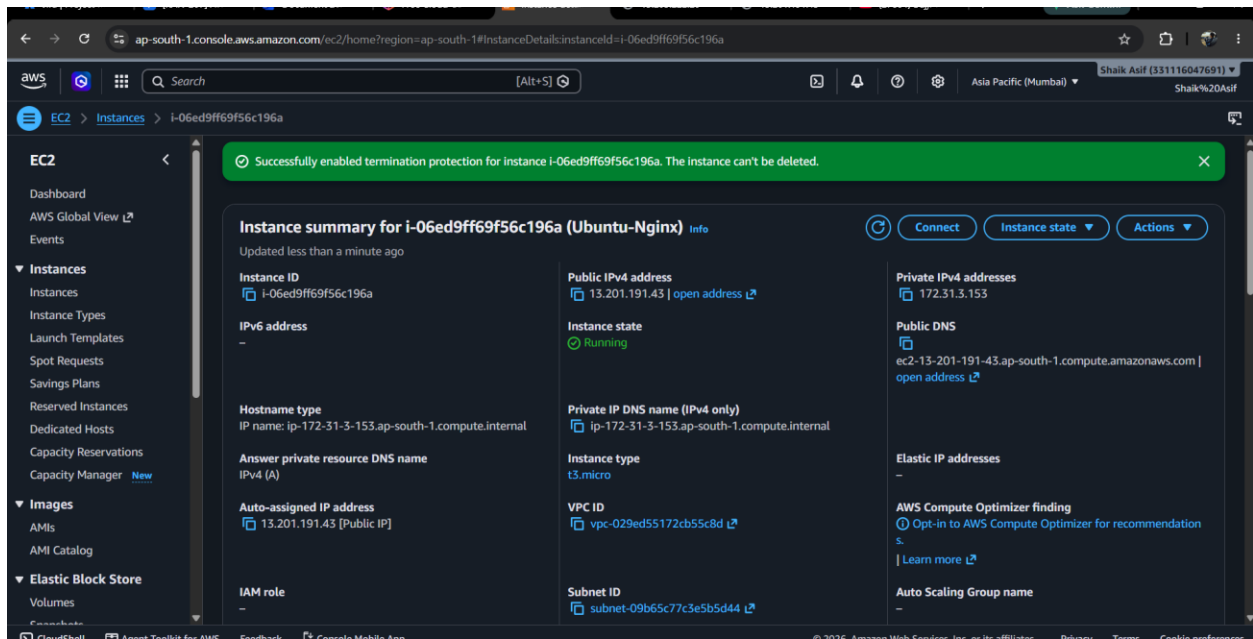
Successfully launched an EC2 instance using the Spot purchasing option. The instance was verified by checking that its lifecycle was shown as spot.

Enable termination policy on the EC2 created in Task 2.

Steps

1. Open **AWS Management Console** → **EC2**.
2. Go to **Instances**.
3. Select the **Ubuntu EC2 instance created in Task 2**.
4. Click **Actions**.
5. Select **Instance settings**.
6. Click **Change termination protection**.
7. Select **Enable**.
8. Click **Save**.
9. Verify that **Termination protection** is enabled for the instance.





Result

Successfully enabled termination protection for the Ubuntu EC2 instance created in Task 2. This prevents the instance from being accidentally terminated.

Launch one EC2 using AWS CLI.

Steps

1. Open **Git Bash** on your Windows system.
2. Verify AWS CLI installation:aws --version
3. Configure AWS CLI:aws configureEnter your AWS credentials, region:ap-south-1and output format:json
4. Verify AWS CLI authentication:aws sts get-caller-identity
5. Find an Amazon Linux AMI:aws ec2 describe-images \
--owners amazon \
--filters "Name=name,Values=al2023-ami-2023*" \
--query "Images | sort_by(@, &CreationDate)[-1].[ImageId,Name]" \
--output table
6. Use the AMI ID obtained from the previous command.
7. Check the available key pairs:aws ec2 describe-key-pairs --query "KeyPairs[*].KeyName" --output table
8. Check the available security groups:aws ec2 describe-security-groups \
--query "SecurityGroups[*].[GroupId,GroupName]" \
--output table
9. Launch the EC2 instance using AWS CLI:aws ec2 run-instances \
--image-id ami-0f48d8eda3141f50a \
--instance-type t3.micro \
--key-name amazon_task \
--security-group-ids sg-03e72047d0168c893 \
--count 1
10. Verify the instance:aws ec2 describe-instances \
--query "Reservations[].Instances[].[InstanceId,State.Name,InstanceType]" \
--output table
11. Confirm that the instance state becomes:running

aws IAM > IAM users

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

- Roles
- Policies
- IAM users**
- IAM user groups
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

Access reports

- Access Analyzer
- Resource analysis
- Unused access
- Analyzer settings

User created successfully

You can view and download the user's password and email instructions for signing in to the AWS Management Console.

View user

IAM users (1) info

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

Search

☐	User name	Path	Group	Last activity	MFA	Password age	Console last sign-in	Acc
☐	asifshaik6558	/	0	-	-	-	-	-

aws IAM > IAM users > asifshaik6558 > Create access key

This is the only time that the secret access key can be viewed or downloaded. You cannot recover it later. However, you can create a new access key any time.

Step 1: Access key best practices & alternatives

Step 2 - optional: Set description tag

Step 3: Retrieve access keys

Retrieve access keys info

Access key

If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key: AKIAU2GAN5VFVM3PUIV2

Secret access key: ***** Show

Access key best practices

- Never store your access key in plain text, in a code repository, or in code.
- Disable or delete access key when no longer needed.
- Enable least-privilege permissions.
- Rotate access keys regularly.

For more details about managing access keys, see the best practices for managing AWS access keys.

Download .csv file Done

```
asifs@Asif_Shaik MINGW64 ~
$ aws --version
aws-cli/2.36.30 Python/3.14.6 Windows/11 exe/AMD64

asifs@Asif_Shaik MINGW64 ~
$ ^C

asifs@Asif_Shaik MINGW64 ~
$ aws configure

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws login'.

AWS Access Key ID [None]: AKIAU2GAN5VFVM3PUIV2
AWS Secret Access Key [None]: j1xZ0wKgwUFQOHff7Jttu4Co8zcUD0MaZEecnPXj
Default region name [None]: ap-south-1
Default output format [None]: json

asifs@Asif_Shaik MINGW64 ~
$ aws sts get-caller-identity
{
  "UserId": "AIDAU2GAN5VFQYEDKTDJ",
  "Account": "331116047691",
  "Arn": "arn:aws:iam::331116047691:user/asifshaik6558"
}
```

```

asifs@Asif_Shaik MINGW64 ~
$ aws ec2 describe-images \
> --owners amazon \
> --filters "Name=name,Values=al2023-ami-2023*" \
> --query "Images | sort_by(@, &CreationDate)[-1].[ImageId,Name]" \
> --output table

```

```

-----
|                               |
| DescribeImages              |
|                               |
+-----+
| ami-0f48d8eda3141f50a      |
| al2023-ami-2023.12.20260817.0-kernel-6.12-x86_64 |
+-----+

```

MINGW64:/c/Users/asifs

```

asifs@Asif_Shaik MINGW64 ~
$ aws ec2 describe-images \
> --owners amazon \
> --filters "Name=name,Values=al2023-ami-2023*" \
> --query "Images | sort_by(@, &CreationDate)[-1].[ImageId,Name]" \
> --output table

```

```

-----
|                               |
| DescribeImages              |
|                               |
+-----+
| ami-0f48d8eda3141f50a      |
| al2023-ami-2023.12.20260817.0-kernel-6.12-x86_64 |
+-----+

```

```

asifs@Asif_Shaik MINGW64 ~
$ aws ec2 describe-key-pairs --query "KeyPairs[*].KeyName" --output table

```

```

-----
| DescribeKeyPairs |
|-----|
| windows_task2   |
| shaik           |
| apple           |
| ubuntu_task     |
| devops-key      |
| amazon_task     |
| banana          |
| purchasing      |
| dev             |
| hero            |
| windows_task    |
| today           |
+-----+

```

```

asifs@Asif_Shaik MINGW64 ~
$ aws ec2 describe-security-groups \
> --query "SecurityGroups[*].[GroupId,GroupName]" \
> --output table

```

```

-----
|                               |
| DescribeSecurityGroups      |
|                               |
+-----+-----+
| sg-0520545dd1fba0f34      | launch-wizard-11 |
| sg-0ff6cfdd486d556b9      | launch-wizard-16 |
| sg-07cac3e042bc89180      | launch-wizard-15 |
| sg-06751e9b99cb422da      | launch-wizard-1  |
| sg-03e72047d0168c893      | default          |
| sg-08c6a892e9fa5fbbc      | launch-wizard-2  |
| sg-0ca8678661aadcf82      | launch-wizard-9  |
| sg-003d91ebd1bab575d      | launch-wizard-12 |
| sg-045fcd3a9a250e050      | launch-wizard-8  |
| sg-0110988b176e575d7      | launch-wizard-3  |
| sg-08ea567573e5bfb27      | launch-wizard-5  |
| sg-0293c69dce0b715ba      | launch-wizard-17 |
| sg-05ac7ad59360063cc      | launch-wizard-4  |
| sg-00ba7165c63154620      | launch-wizard-10 |
| sg-0ea8a97400cef7e87      | launch-wizard-7  |
| sg-003d6fce7727b478a      | launch-wizard-13 |
| sg-0f953193d06eaf220      | launch-wizard-6  |
| sg-0922141ef8fcd262f      | launch-wizard-14 |
+-----+-----+

```

```

MINGW64/c/Users/asifs
asifs@Asif-Shaik MINGW64 ~
$ aws ec2 run-instances \
> --image-id ami-0f48d8eda3141f50a \
> --instance-type t3.micro \
> --key-name amazon_task \
> --security-group-ids sg-03e72047d0168c893 \
> --count 1
{
  "ReservationId": "r-0bdf96e0ca97ab898",
  "OwnerId": "331116047691",
  "Groups": [],
  "Instances": [
    {
      "Architecture": "x86_64",
      "BlockDeviceMappings": [],
      "ClientToken": "6aad3d92-f620-48e6-b7e2-96c47f46c819",
      "EbsOptimized": false,
      "EnaSupport": true,
      "Hypervisor": "xen",
      "NetworkInterfaces": [
        {
          "Attachment": {
            "AttachTime": "2026-08-25T17:23:54+00:00",
            "AttachmentId": "eni-attach-0a9e8d4114707e0ff",
            "DeletedOnTermination": true,
            "DeviceIndex": 0,
            "Status": "attaching",
            "NetworkCardIndex": 0
          },
          "Description": "",
          "Groups": [
            {
              "GroupId": "sg-03e72047d0168c893",
              "GroupName": "default"
            }
          ],
          "Ipv6Addresses": [],
          "MacAddress": "0a:8f:32:ad:b2:93",
          "NetworkInterfaceId": "eni-0646f5f18357269fc",
          "OwnerId": "331116047691",
          "PrivateDnsName": "ip-172-31-4-140.ap-south-1.compute.internal",
          "PrivateIpAddress": "172.31.4.140",
          "PrivateIpAddresses": [
            {
              "Primary": true,
              "PrivateDnsName": "ip-172-31-4-140.ap-south-1.compute.internal",
              "PrivateIpAddress": "172.31.4.140"
            }
          ],
          "SourceDestCheck": true,
          "Status": "in-use",
          "SubnetId": "subnet-09b65c77c3e5b5d44",
          "VpcId": "vpc-029ed55172cb55c8d",
          "InterfaceType": "Interface"
        }
      ],
      "SourceDestCheck": true,
      "Status": "in-use",
      "SubnetId": "subnet-09b65c77c3e5b5d44",
      "VpcId": "vpc-029ed55172cb55c8d",
      "InterfaceType": "Interface"
    }
  ]
}

```

```

    "Code": 0,
    "Name": "pending"
  },
  "PrivateDnsName": "ip-172-31-4-140.ap-south-1.compute.internal",
  "PublicDnsName": "",
  "StateTransitionReason": "",
  "KeyName": "amazon_task",
  "AmiLaunchIndex": 0,
  "ProductCodes": [],
  "InstanceType": "t3.micro",
  "LaunchTime": "2026-08-25T17:23:54+00:00",
  "Placement": {
    "AvailabilityZoneId": "aps1-az3",
    "GroupName": "",
    "Tenancy": "default",
    "AvailabilityZone": "ap-south-1b"
  },
  "Monitoring": {
    "State": "disabled"
  },
  "SubnetId": "subnet-09b65c77c3e5b5d44",
  "VpcId": "vpc-029ed55172cb55c8d",
  "PrivateIpAddress": "172.31.4.140"
}
]

asifs@Asif-Shaik MINGW64 ~
$ aws ec2 describe-instances \
> --query "Reservations[].Instances[].[InstanceId,State.Name,InstanceType]" \
> --output table
-----
DescribeInstances
-----
InstanceId | State.Name | InstanceType
-----
i-0e65301902dfa9782 | stopped | t3.micro
i-07fd12b3c32dcf99 | stopped | t3.micro
i-012dec17ebaef805 | stopped | t3.micro
i-0f742bcb523e4f95b | stopped | t3.micro
i-06ad27df6b030bab9 | stopped | t3.micro
i-0a2f69d0c734159c1 | stopped | t3.micro
i-088ca479fb343045d | stopped | t3.micro
i-00492db32721b2053 | stopped | t3.micro
i-0f67986fe07a767b7 | stopped | t3.micro
i-018c87a8279be7035 | stopped | t3.micro
i-0338057412f8a859f | stopped | t3.micro
i-05df117383d57f2d2 | stopped | t3.small
i-06ed9ff69f56c196a | running | t3.micro
i-0f6d35ee94371ebe6 | running | t3.micro
i-0040d988d1f381a8 | stopped | t3.micro
i-00ca77d4db28281a4 | running | t3.micro
i-064df171fa2ff9412 | running | t3.micro
i-089376c3c3589cd14 | running | t3.micro
-----

```

Result

Successfully launched one EC2 instance using AWS CLI. The instance was verified using the describe-instances command and its state was confirmed as running.